

Temporal Superposition in GPT-2: How Temporal Time is Encoded and Distributed Across Layers

Sam Brown Peyton Calvert

Your Institution

{sbrown, pcalvert}@yourinstitution.edu

Abstract

We investigate how GPT-2-small encodes historical time in its residual stream using sparse autoencoders (SAEs). TODO: fill in findings summary.

1 Introduction

Large language models have demonstrated their ability to "know" historical facts, but we have limited understanding of how that knowledge is stored internally. Understanding the geometric structure of factual knowledge in LLMs is a foundational question for mechanistic interpretability.

2 Related Work

TODO

3 Background

3.1 Residual Stream

Transformer models process tokens by passing representations through a sequence of attention and MLP layers. These layers write incremental updates into a shared vector called the residual stream. Formally if $\mathbf{x}^{(\ell)} \in \mathbb{R}^d$ denotes the residual stream at layer ℓ , each layer computes $\mathbf{x}^{(\ell+1)} = \mathbf{x}^{(\ell)} + f^{(\ell)}$, where $f^{(\ell)}$ is the combined attention and MLP transformation. This additive structure means the residual stream accumulates information across depth, and that representations at any layer can be read as a type of superposition of all contributions made so far. In this work we probe the residual stream of GPT-2 Small Radford et al. [2019], a 117 million parameter transformer with a hidden dimension of $d = 768$.

3.2 Superposition

A naive expectation is that each dimension of the residual stream encodes one interpretable feature. In practice, the number of concepts a model must represent exceeds its hidden

dimension d ($\mathbb{R}^{768}$ in this paper). As a result, models exploit the fact that high dimensional spaces can accommodate many nearly-orthogonal directions. This phenomenon is called superposition and it allows a model to store a large amount of features at the same time. Features in superposition are distributed across many neurons.

3.3 Sparse Autoencoders

Sparse autoencoders (SAEs) offer a way to recover the underlying precise features from an ambiguous residual stream Cunningham et al. [2024]. An SAE consists of an encoder that projects the d -dimensional activation into a much larger latent space $\mathbb{R}^{d_{\text{SAE}}}$ with $d_{\text{SAE}} \gg d$, followed by a ReLU that enforces sparsity and a linear decoder that reconstructs the original activation. Training minimises reconstruction error subject to an L_1 penalty on the latent activations, encouraging each input to be explained by only a small number of active features at a time. The resulting latent dimensions have been shown to correspond to interpretable, monosemantic concepts Cunningham et al. [2024], Bloom [2024]. In this work we use the SAE of Bloom [2024] trained on GPT-2 Small’s residual stream.

4 Method

4.1 Model and SAE

We do all experiments on GPT-2 Small Radford et al. [2019], a decoder-only transformer with 12 layers, 12 attention heads, and a residual stream dimension of $d = 768$, totalling approximately 117 million parameters. GPT-2 Small is a well-studied model in the context of mechanistic interpretability, making it a good model for probing internal representations.

To decompose residual stream activations, we use the suite of SAEs released by Bloom [2024], trained on GPT-2 Small’s residual stream under the release identifier `gpt2-small-res-jb`. Each SAE is trained on the `hook_resid_pre` activation at its corresponding layer – that is, the residual stream state *before* the attention and MLP transformations of that layer have been applied. The SAE architecture uses a linear encoder with ReLU activation followed by a linear decoder, with an expansion factor of $64\times$, giving a latent dimension of $d_{\text{SAE}} = 49,152$. For each layer ℓ we load the corresponding SAE independently, allowing us to probe how feature representations evolve across depth.

All activations are extracted at the final token position of each prompt, as this is the position at which the model has processed the full input and is most likely to have integrated contextual information into a coherent representation.

4.2 Prompt Design

TODO (May still try different prompts like prof mentioned... come back to)

4.3 Temporal Axis Measurement

TODO

4.4 Superposition Depth Scan

TODO

4.5 Circular Encoding Measurement

TODO

5 Experiments and Results

5.1 BC/AD Token Contamination

TODO

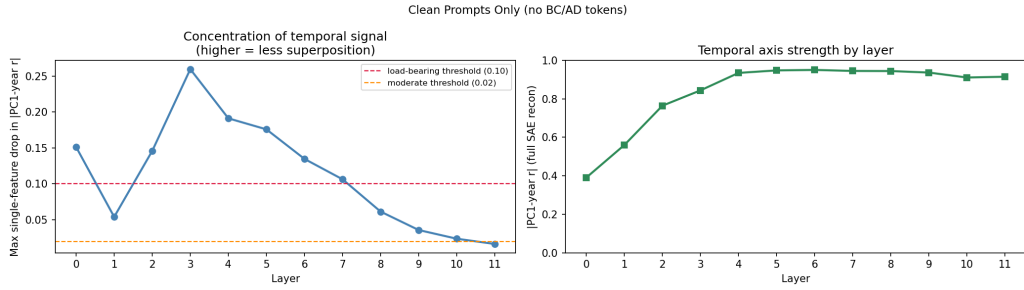


Figure 1: Left: concentration of temporal signal (max single-feature SAE ablation drop) across all 12 layers for clean prompts. Right: baseline temporal axis strength ($|PC1\text{-year } r|$) across layers. The temporal axis builds progressively from near zero at layer 0 to $|r| \approx 0.93$ by layer 4, then remains stable.

5.2 Temporal Axis Construction

TODO

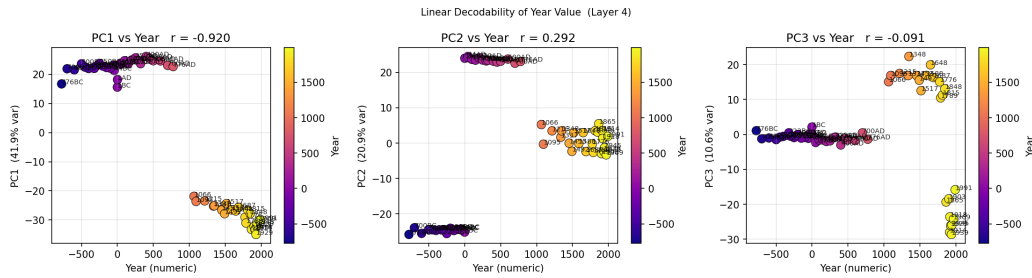


Figure 2: PC1 vs year value for all historical prompts at layer 4, showing strong linear decodability ($r = -0.920$).

5.3 Superposition Transition

TODO

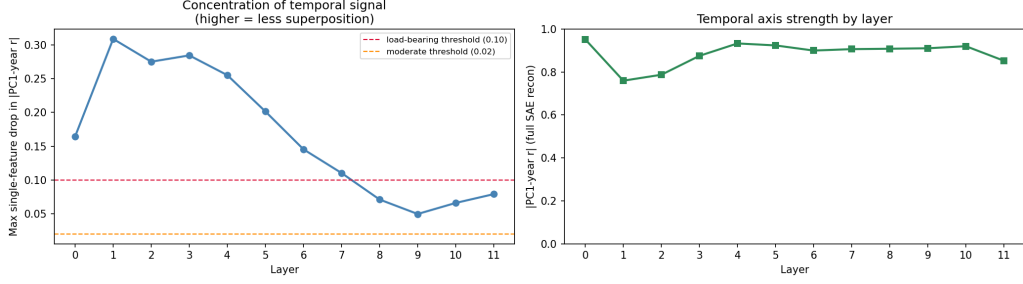


Figure 3: Superposition depth scan on original dataset across all 12 layers. Left: max single-feature drop. Right: baseline $|r|$. Note early layers dominated by BC token detector (feature 22992).

5.4 Load-Bearing Feature Identification

TODO

5.5 Circular Concepts: Days of Week and Months of Year

TODO

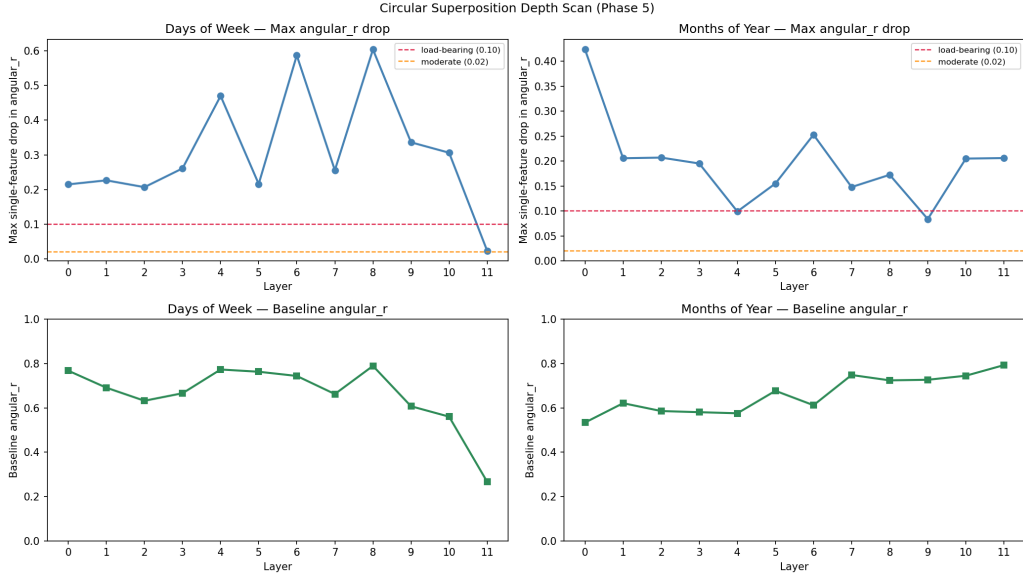


Figure 4: Superposition depth scan for days of the week and months of the year using angular correlation as the signal metric. Unlike linear temporal encoding, circular concepts show oscillating concentration with no monotonic decline.

5.6 Days of the Month: Novel Geometric Analysis

TODO

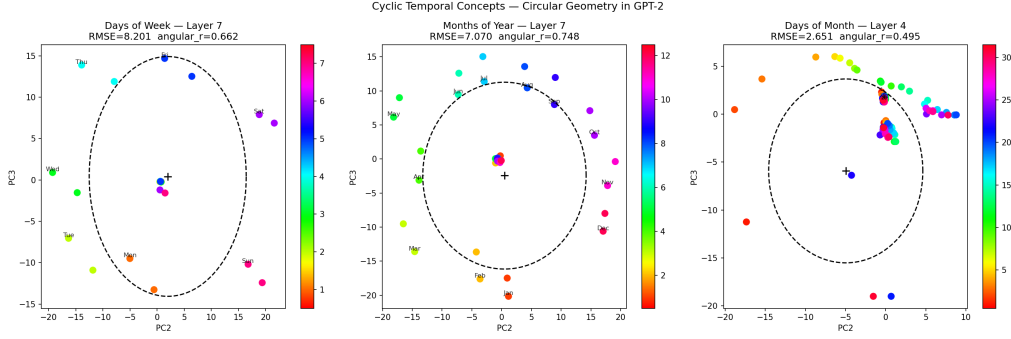


Figure 5: PC2 vs PC3 geometric structure for three cyclic temporal concepts in GPT-2-small. Left: days of the week (layer 7). Middle: months of the year (layer 7). Right: days of the month (best layer). Points colored by position in cycle using HSV colormap. Dashed circle shows algebraic least-squares fit.

6 Discussion

6.1 Linear vs Circular Temporal Encoding

TODO

6.2 The BC/AD Contamination Finding

TODO

6.3 Limitations

TODO

7 Conclusion

TODO

Acknowledgements and Contributions

Sam Brown: TODO (e.g. data pipeline, phase 1-3 analysis, writing)

Peyton Calvert: TODO (e.g. geometric analysis, phase 5-6, writing)

Questions for the Instructor

1. TODO
2. TODO

References

- J. Bloom. Gpt2 residual stream sparse autoencoders. <https://huggingface.co/jbloom/GPT2-Small-SAEs-Reformatted>, 2024. SAE release on Hugging Face, accessed 2024.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2309.08600>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

A Additional Figures

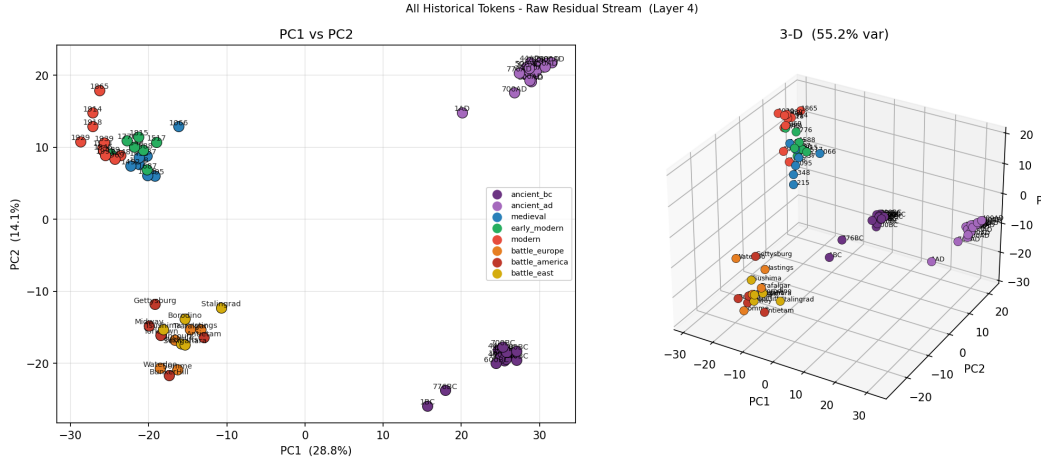


Figure 6: PCA of raw residual stream activations at layer 4 for all historical prompts. BC years (dark purple) and modern years (red/orange) separate clearly along PC1 and PC2.

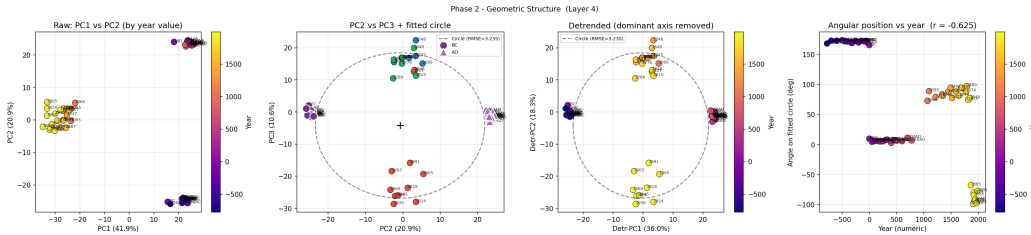


Figure 7: Phase 2 geometric analysis at layer 4. Left to right: raw PCA, PC2 vs PC3 with fitted circle (RMSE=3.235), detrended PCA, angular position vs year ($r = -0.625$). The high circle RMSE indicates the helical/circular hypothesis is not supported for historical time.

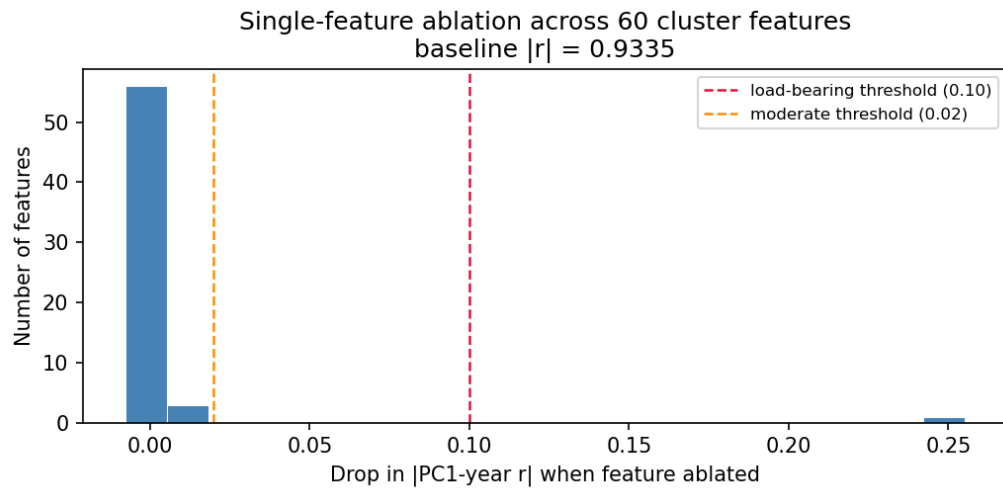


Figure 8: Distribution of single-feature ablation drops across 60 cluster features at layer 4. One feature causes a drop of ~ 0.25 , while the remaining 59 cause drops near zero.